

Data Structures

Lecture 13: Basic Sorting Algorithms

Soobin Um

Apr. 29, 2026

Sorting (정렬)

- 리스트의 아이템들을 일정 기준(예: 오름차순, 내림차순)에 따라
순서화하는 작업

Sorting (정렬)

- 리스트의 아이템들을 일정 기준(예: 오름차순, 내림차순)에 따라
순서화하는 작업
- 정렬의 필요성
 - 탐색 성능 향상
 - 이진 탐색 등의 더 빠른 검색 알고리즘을 적용할 수 있음
 - 데이터 분석을 위한 기반
 - 중앙값, 분위수, 이상치 (outlier) 탐지가 쉽고 정확해짐
 - UI/UX 향상
 - 프로그램 또는 서비스를 이용하는 사용자가 원하는 정보를 더 빠르게 얻을 수 있음
예) 온라인 쇼핑몰 상품을 가격순 또는 인기순으로 정렬
 - 이 밖에도 컴퓨터 과학의 여러 분야에서 가장 기초적으로 행해지는 작업 중의 하나

Sorting Algorithms

- 정렬 알고리즘의 종류
 - **Basic:** Bubble sort, Selection sort, Insertion sort
 - 직관적이지만 다소 비효율적
 - **Advanced:** Quick sort, Merge sort, Heap sort
 - 분할정복 전략과 효율적인 자료 구조를 활용하여 정렬 속도를 크게 개선

Sorting Algorithms

- 정렬 알고리즘의 종류

- **Basic:** Bubble sort, Selection sort, Insertion sort

- 직관적이지만 다소 비효율적

- **Advanced:** Quick sort, Merge sort, Heap sort

- 분할정복 전략과 효율적인 자료 구조를 활용하여 정렬 속도를 크게 개선

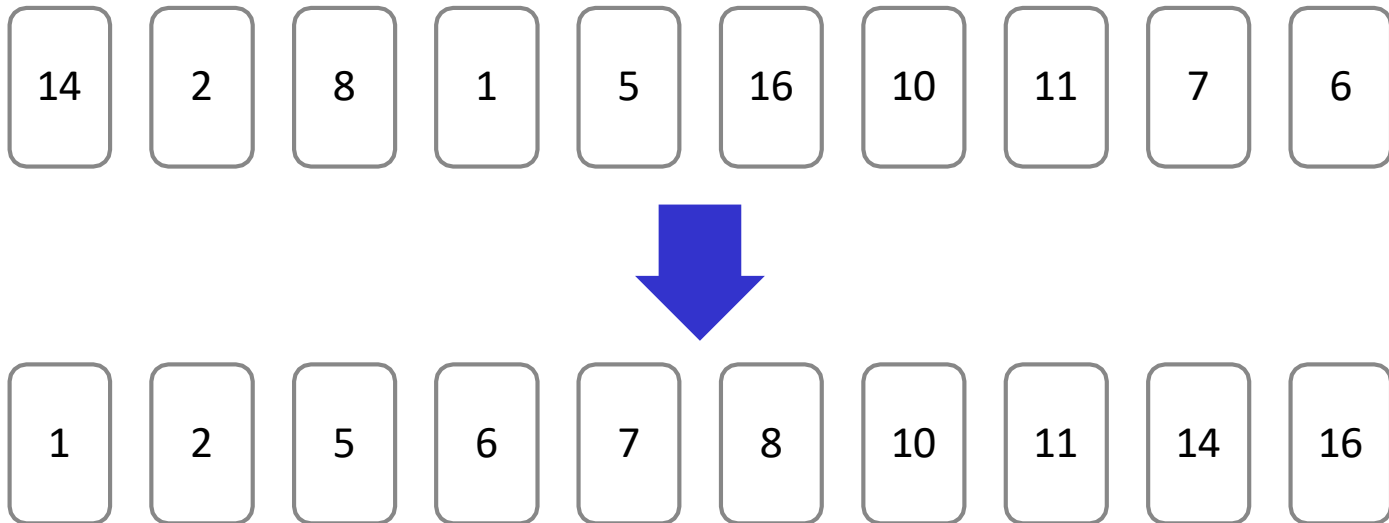
- 비용 측정 기준

- 비교 연산 횟수

- 데이터 교환 (스왑) 횟수

Basic Sorting Algorithms

- 세 가지 기본 정렬 알고리즘의 컨셉
 - **Bubble sort:** “두 개씩 보고 큰 것을 오른쪽으로 보내자”
 - **Selection sort:** “가장 작은 것을 가장 왼쪽에 놓자”
 - **Insertion sort:** “일단 하나를 잡고, 적절한 위치에 넣자”



Bubble Sort

Bubble Sort

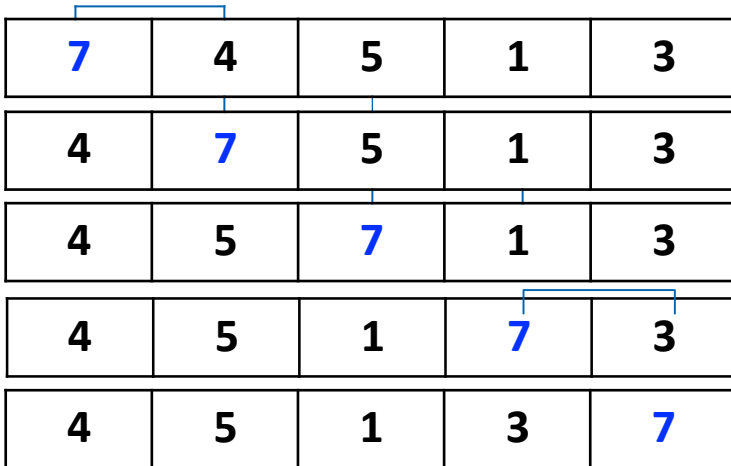
- 큰 값이 계속 오른쪽 끝으로 밀려가며 한 단계씩 정렬된 영역이 증가

<https://youtu.be/nmhjrl-aW5o>

Bubble Sort: 예제

- 큰 값이 계속 오른쪽 끝으로 밀려가며 한 단계씩 정렬된 영역이 증가

1회전: 가장 큰 값이 맨 오른쪽으로 이동



7	4	5	1	3
4	7	5	1	3
4	5	7	1	3
4	5	1	7	3
4	5	1	3	7

Bubble Sort: 예제

- 큰 값이 계속 오른쪽 끝으로 밀려가며 한 단계씩 정렬된 영역이 증가

1회전: 가장 큰 값이 맨 오른쪽으로 이동

7	4	5	1	3
4	7	5	1	3
4	5	7	1	3
4	5	1	7	3
4	5	1	3	7

2회전: 두 번째로 큰 값이 오른쪽에서 두 번째로 이동

4	5	1	3	7
4	5	1	3	7
4	1	5	3	7
4	1	3	5	7

Bubble Sort: 예제

- 큰 값이 계속 오른쪽 끝으로 밀려가며 한 단계씩 정렬된 영역이 증가

1회전: 가장 큰 값이 맨 오른쪽으로 이동

7	4	5	1	3
4	7	5	1	3
4	5	7	1	3
4	5	1	7	3
4	5	1	3	7

2회전: 두 번째로 큰 값이 오른쪽에서 두 번째로 이동

4	5	1	3	7
4	5	1	3	7
4	1	5	3	7
4	1	3	5	7

3회전: 세 번째로 큰 값이 오른쪽에서 세 번째로 이동

4	1	3	5	7
1	4	3	5	7
1	3	4	5	7

Bubble Sort: 예제

- 큰 값이 계속 오른쪽 끝으로 밀려가며 한 단계씩 정렬된 영역이 증가

1회전: 가장 큰 값이 맨 오른쪽으로 이동

7	4	5	1	3
4	7	5	1	3
4	5	7	1	3
4	5	1	7	3
4	5	1	3	7

2회전: 두 번째로 큰 값이 오른쪽에서 두 번째로 이동

4	5	1	3	7
4	5	1	3	7
4	1	5	3	7
4	1	3	5	7

3회전: 세 번째로 큰 값이 오른쪽에서 세 번째로 이동

4	1	3	5	7
1	4	3	5	7
1	3	4	5	7

4회전: 네 번째로 큰 값이 오른쪽에서 네 번째로 이동

1	3	4	5	7
---	---	---	---	---

Bubble Sort: 예제

- 큰 값이 계속 오른쪽 끝으로 밀려가며 한 단계씩 정렬된 영역이 증가

1회전: 가장 큰 값이 맨 오른쪽으로 이동

7	4	5	1	3
4	7	5	1	3
4	5	7	1	3
4	5	1	7	3
4	5	1	3	7

2회전: 두 번째로 큰 값이 오른쪽에서 두 번째로 이동

4	5	1	3	7
4	5	1	3	7
4	1	5	3	7
4	1	3	5	7

3회전: 세 번째로 큰 값이 오른쪽에서 세 번째로 이동

4	1	3	5	7
1	4	3	5	7
1	3	4	5	7

4회전: 네 번째로 큰 값이 오른쪽에서 네 번째로 이동

1	3	4	5	7
---	---	---	---	---



1	3	4	5	7
---	---	---	---	---

총 10회의 비교 연산, 8회의 교환 (swap) 발생

Bubble Sort: 알고리즘 및 연산 횟수

- 큰 값이 계속 오른쪽 끝으로 밀려가며 한 단계씩 정렬된 영역이 증가

```
void bubbleSort(List<Integer> list) {  
    for (int i = 0; i < list.size() - 1; i++) {  
        for (int j = 0; j < list.size() - 1 - i; j++) {  
            if (list.get(j) > list.get(j + 1)) {  
                swap(list, j, j + 1);  
            }  
        }  
    }  
}
```

Bubble Sort: 알고리즘 및 연산 횟수

- 큰 값이 계속 오른쪽 끝으로 밀려가며 한 단계씩 정렬된 영역이 증가

```
void bubbleSort(List<Integer> list) {  
    for (int i = 0; i < list.size() - 1; i++) {  
        for (int j = 0; j < list.size() - 1 - i; j++) {  
            if (list.get(j) > list.get(j + 1)) {  
                swap(list, j, j + 1);  
            }  
        }  
    }  
}
```

- 연산 횟수
 - Best: 0 swaps, $n^2/2$ comparisons
 - Worst: $n^2/2$ swaps, $n^2/2$ comparisons

Selection Sort

Selection Sort

- 매 회전마다 정렬되지 않은 구간에서 가장 작은 값을 선택한 뒤, 해당 구간의 맨 앞 원소와 교환하는 작업을 반복

<https://youtu.be/xWBP4lzkoyM>

Selection Sort: 예제

- 매 회전마다 정렬되지 않은 구간에서 가장 작은 값을 선택한 뒤, 해당 구간의 맨 앞 원소와 교환하는 작업을 반복

1회전: 가장 작은 값이 왼쪽에서 첫 번째로 이동

7	4	5	1	3
1	4	5	7	3

Selection Sort: 예제

- 매 회전마다 정렬되지 않은 구간에서 가장 작은 값을 선택한 뒤, 해당 구간의 맨 앞 원소와 교환하는 작업을 반복

1회전: 가장 작은 값이 왼쪽에서 첫 번째로 이동

7	4	5	1	3
1	4	5	7	3

2회전: 두 번째로 작은 값이 왼쪽에서 두 번째로 이동

1	4	5	7	3
1	3	5	7	4

Selection Sort: 예제

- 매 회전마다 정렬되지 않은 구간에서 가장 작은 값을 선택한 뒤, 해당 구간의 맨 앞 원소와 교환하는 작업을 반복

1회전: 가장 작은 값이 왼쪽에서 첫 번째로 이동

7	4	5	1	3
1	4	5	7	3

2회전: 두 번째로 작은 값이 왼쪽에서 두 번째로 이동

1	4	5	7	3
1	3	5	7	4

3회전: 세 번째로 작은 값이 왼쪽에서 세 번째로 이동

1	3	5	7	4
1	3	4	7	5

Selection Sort: 예제

- 매 회전마다 정렬되지 않은 구간에서 가장 작은 값을 선택한 뒤, 해당 구간의 맨 앞 원소와 교환하는 작업을 반복

1회전: 가장 작은 값이 왼쪽에서 첫 번째로 이동

7	4	5	1	3
1	4	5	7	3

2회전: 두 번째로 작은 값이 왼쪽에서 두 번째로 이동

1	4	5	7	3
1	3	5	7	4

3회전: 세 번째로 작은 값이 왼쪽에서 세 번째로 이동

1	3	5	7	4
1	3	4	7	5

4회전: 네 번째로 작은 값이 왼쪽에서 네 번째로 이동

1	3	4	7	5
1	3	4	5	7

Selection Sort: 예제

- 매 회전마다 정렬되지 않은 구간에서 가장 작은 값을 선택한 뒤, 해당 구간의 맨 앞 원소와 교환하는 작업을 반복

1회전: 가장 작은 값이 왼쪽에서 첫 번째로 이동

7	4	5	1	3
1	4	5	7	3

2회전: 두 번째로 작은 값이 왼쪽에서 두 번째로 이동

1	4	5	7	3
1	3	5	7	4

3회전: 세 번째로 작은 값이 왼쪽에서 세 번째로 이동

1	3	5	7	4
1	3	4	7	5

4회전: 네 번째로 작은 값이 왼쪽에서 네 번째로 이동

1	3	4	7	5
1	3	4	5	7



1	3	4	5	7
---	---	---	---	---

총 10회의 비교 연산, 4회의 교환 (swap) 발생

Selection Sort: 알고리즘 및 연산 횟수

- 매 회전마다 정렬되지 않은 구간에서 가장 작은 값을 선택한 뒤, 해당 구간의 맨 앞 원소와 교환하는 작업을 반복

```
void selectionSort(List<Integer> list) {  
    int n = list.size();  
  
    for (int i = 0; i < n - 1; i++) {  
        int minIndex = i;  
  
        // 정렬되지 않은 부분에서 최솟값 탐색  
        for (int j = i + 1; j < n; j++)  
            if (list.get(j) < list.get(minIndex))  
                minIndex = j;  
  
        // 찾은 최솟값을 i 위치로 이동  
        swap(list, i, minIndex);  
    }  
}
```

Selection Sort: 알고리즘 및 연산 횟수

- 매 회전마다 정렬되지 않은 구간에서 가장 작은 값을 선택한 뒤, 해당 구간의 맨 앞 원소와 교환하는 작업을 반복

```
void selectionSort(List<Integer> list) {  
    int n = list.size();  
  
    for (int i = 0; i < n - 1; i++) {  
        int minIndex = i;  
  
        // 정렬되지 않은 부분에서 최솟값 탐색  
        for (int j = i + 1; j < n; j++)  
            if (list.get(j) < list.get(minIndex))  
                minIndex = j;  
  
        // 찾은 최솟값을 i 위치로 이동  
        swap(list, i, minIndex);  
    }  
}
```

- 연산 횟수
 - Best: 0 swaps, $n^2/2$ comparisons
 - Worst: $n-1$ swaps, $n^2/2$ comparisons

Selection Sort: 알고리즘 및 연산 횟수

- 매 회전마다 정렬되지 않은 구간에서 가장 작은 값을 선택한 뒤, 해당 구간의 맨 앞 원소와 교환하는 작업을 반복

```
void selectionSort(List<Integer> list) {  
    int n = list.size();  
  
    for (int i = 0; i < n - 1; i++) {  
        int minIndex = i;  
  
        // 정렬되지 않은 부분에서 최솟값 탐색  
        for (int j = i + 1; j < n; j++)  
            if (list.get(j) < list.get(minIndex))  
                minIndex = j;  
  
        // 찾은 최솟값을 i 위치로 이동  
        swap(list, i, minIndex);  
    }  
}
```

- 연산 횟수

- Best: 0 swaps, $n^2/2$ comparisons
- Worst: $n-1$ swaps, $n^2/2$ comparisons

→ 버블 정렬보다 swap 횟수가 더 적음

Insertion Sort

Insertion Sort

- 매 회전마다 현재 원소를 꺼내어, **앞쪽 정렬된 구간에서 알맞은 위치를 찾아 삽입**하는 작업을 반복

<https://youtu.be/OGzPmgsl-pQ>

Insertion Sort: 예제

- 매 회전마다 현재 원소를 꺼내어, **앞쪽 정렬된 구간에서 알맞은 위치를 찾아 삽입**하는 작업을 반복

1회전 (i=1)

- 삽입 대상: 4, 왼쪽 정렬 구간: [7]

7	4	5	1	3
4	7	5	1	3

1회의 비교 연산, 1회의 교환 (swap) 발생

Insertion Sort: 예제

- 매 회전마다 현재 원소를 꺼내어, **앞쪽 정렬된 구간에서 알맞은 위치를 찾아 삽입**하는 작업을 반복

1회전 (i=1)

- 삽입 대상: 4, 왼쪽 정렬 구간: [7]

7	4	5	1	3
4	7	5	1	3

1회의 비교 연산, 1회의 교환 (swap) 발생

2회전 (i=2)

- 삽입 대상: 5, 왼쪽 정렬 구간: [4, 7]

4	7	5	1	3
4	5	7	1	3

2회의 비교 연산, 1회의 교환 (swap) 발생

Insertion Sort: 예제

- 매 회전마다 현재 원소를 꺼내어, **앞쪽 정렬된 구간에서 알맞은 위치를 찾아 삽입**하는 작업을 반복

1회전 (i=1)

- 삽입 대상: 4, 왼쪽 정렬 구간: [7]

7	4	5	1	3
4	7	5	1	3

1회의 비교 연산, 1회의 교환 (swap) 발생

2회전 (i=2)

- 삽입 대상: 5, 왼쪽 정렬 구간: [4, 7]

4	7	5	1	3
4	5	7	1	3

2회의 비교 연산, 1회의 교환 (swap) 발생

3회전 (i=3)

- 삽입 대상: 1, 왼쪽 정렬 구간: [4, 5, 7]

4	5	7	1	3
1	4	5	7	3

3회의 비교 연산, 3회의 교환 (swap) 발생

Insertion Sort: 예제

- 매 회전마다 현재 원소를 꺼내어, **앞쪽 정렬된 구간에서 알맞은 위치를 찾아 삽입**하는 작업을 반복

1회전 (i=1)

- 삽입 대상: 4, 왼쪽 정렬 구간: [7]

7	4	5	1	3
4	7	5	1	3

1회의 비교 연산, 1회의 교환 (swap) 발생

2회전 (i=2)

- 삽입 대상: 5, 왼쪽 정렬 구간: [4, 7]

4	7	5	1	3
4	5	7	1	3

2회의 비교 연산, 1회의 교환 (swap) 발생

3회전 (i=3)

- 삽입 대상: 1, 왼쪽 정렬 구간: [4, 5, 7]

4	5	7	1	3
1	4	5	7	3

3회의 비교 연산, 3회의 교환 (swap) 발생

4회전 (i=4)

- 삽입 대상: 3, 왼쪽 정렬 구간: [1, 4, 5, 7]

1	4	5	7	3
1	3	4	5	7

4회의 비교 연산, 3회의 교환 (swap) 발생

Insertion Sort: 예제

- 매 회전마다 현재 원소를 꺼내어, **앞쪽 정렬된 구간에서 알맞은 위치를 찾아 삽입**하는 작업을 반복

1회전 (i=1)

- 삽입 대상: 4, 왼쪽 정렬 구간: [7]

7	4	5	1	3
4	7	5	1	3

1회의 비교 연산, 1회의 교환 (swap) 발생

2회전 (i=2)

- 삽입 대상: 5, 왼쪽 정렬 구간: [4, 7]

4	7	5	1	3
4	5	7	1	3

2회의 비교 연산, 1회의 교환 (swap) 발생

3회전 (i=3)

- 삽입 대상: 1, 왼쪽 정렬 구간: [4, 5, 7]

4	5	7	1	3
1	4	5	7	3

3회의 비교 연산, 3회의 교환 (swap) 발생

4회전 (i=4)

- 삽입 대상: 3, 왼쪽 정렬 구간: [1, 4, 5, 7]

1	4	5	7	3
1	3	4	5	7

4회의 비교 연산, 3회의 교환 (swap) 발생



1	3	4	5	7
---	---	---	---	---

총 10회의 비교 연산, 8회의 교환 (swap) 발생

Insertion Sort: 알고리즘 및 연산 횟수

- 매 회전마다 현재 원소를 꺼내어, **앞쪽 정렬된 구간에서 알맞은 위치를 찾아 삽입**하는 작업을 반복

```
void insertSort(List<Integer> list) {  
    for (int i = 1; i < list.length; i++) {  
        for (int j = i; j > 0 && list[j] < list[j - 1]; j--) {  
  
            // swap(list, j, j-1)과 동일  
            int temp = list[j];  
            list[j] = list[j - 1];  
            list[j - 1] = temp;  
        }  
    }  
}
```

Insertion Sort: 알고리즘 및 연산 횟수

- 매 회전마다 현재 원소를 꺼내어, **앞쪽 정렬된 구간에서 알맞은 위치를 찾아 삽입**하는 작업을 반복

```
void insertSort(List<Integer> list) {  
    for (int i = 1; i < list.length; i++) {  
        for (int j = i; j > 0 && list[j] < list[j - 1]; j--) {  
  
            // swap(list, j, j-1)과 동일  
            int temp = list[j];  
            list[j] = list[j - 1];  
            list[j - 1] = temp;  
        }  
    }  
}
```

- 연산 횟수
 - Best: 0 swaps, **$n-1$** comparisons
 - Worst: **$n^2/2$** swaps, **$n^2/2$** comparisons

Insertion Sort: 알고리즘 및 연산 횟수

- 매 회전마다 현재 원소를 꺼내어, **앞쪽 정렬된 구간에서 알맞은 위치를 찾아 삽입**하는 작업을 반복

```
void insertSort(List<Integer> list) {  
    for (int i = 1; i < list.length; i++) {  
        for (int j = i; j > 0 && list[j] < list[j - 1]; j--) {  
  
            // swap(list, j, j-1)과 동일  
            int temp = list[j];  
            list[j] = list[j - 1];  
            list[j - 1] = temp;  
        }  
    }  
}
```

- 연산 횟수
 - Best: 0 swaps, **$n-1$** comparisons
 - Worst: **$n^2/2$** swaps, **$n^2/2$** comparisons
- 거의 정렬되어 있는 (Best case에 가까운) 경우, 비교 연산 횟수가 매우 적음

Summary

알고리즘	전략	시간 복잡도 (비교 횟수 기준)	특징
Bubble Sort	큰 값을 뒤로 이동	$O(n^2)$	Swap 연산이 많음
Selection Sort	작은 값을 앞으로 이동	$O(n^2)$	Swap 연산이 적음
Insertion Sort	정렬 구간에 삽입	$O(n^2)$	거의 정렬되어 있는 리스트에서 매우 효과적

Look ahead

Merge Sort